

Document number	P3741R0
Date	2025-06-21
Audience	LEWG, SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

views::set_operations

- Abstract
- Revision history
- Motivation
- Design
- Implementation experience
- Wording
- References

Abstract

This paper proposes four range adaptors for set operations rated as Tier 3 in [P2760](#), namely `views::set_difference`, `views::set_intersection`, `views::set_union`, and `views::set_symmetric_difference`, to extend the breadth of Ranges.

These adaptors complement existing set algorithms by enabling composable, lazy, and allocation-free views for common set operations. They fill a notable gap in the Ranges library for many practical applications.

Revision history

R0

Initial revision.

Motivation

Although there are corresponding constrained algorithm versions of set operations, they all need to output the results to some sort of output range. This brings advantages of the view's lazy evaluation: we can construct set elements on the fly without allocating memory in advance.

Given that set operations are extremely common in the real world, introducing corresponding range adaptors is valuable and facilitates the user experience with Ranges:

```
/* algorithm approach */
std::vector<int> diff;
ranges::set_difference(v1, v2, std::back_inserter(diff));

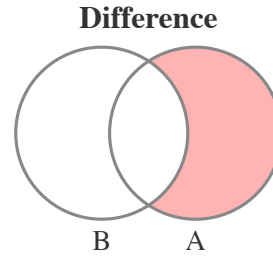
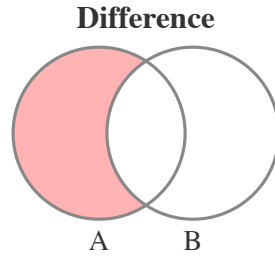
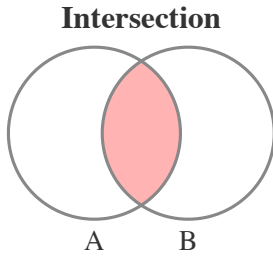
/* view approach, no allocation, composable */
auto diff = v1 | views::set_difference(v2);
```

Design

Pipe syntax support

It is reasonable that `views::set_operations` should be range adaptor objects instead of just CPOs. This allows expressions like `rng1 | views::set_operations(rng2)`, meaning to extract elements from `rng1` through the set operation with `rng2`, which is intuitive and worth supporting.

Constraint for `set_[intersection|difference]_view`

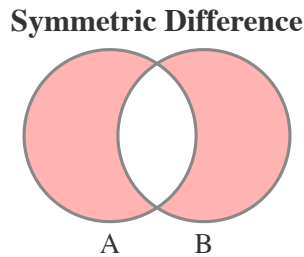
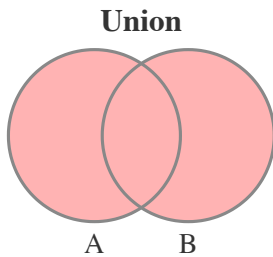


For `views::set_operations(A, B)`, both intersection and difference produce only elements from A, so the result is just a subset of A; the element type of B is irrelevant for output, only its order matters. The only thing that matters is making sure the elements of both ranges are strictly weakly ordered so that they can be compared meaningfully.

The standard already has a concept for this, namely `indirect_strict_weak_order`, which is also used for the corresponding constrained algorithm (a component of `mergeable` concept). The signatures of the two classes would be:

```
template<view V1, view V2>
    requires input_range<V1> && input_range<V2> &&
               indirect_strict_weak_order<ranges::less, iterator_t<V1>, iterator_t<V2>>
    class set_[intersection|difference]_view;
```

Constraint for `set_[union|symmetric_difference]_view`



Unlike the above, union and symmetric difference produce elements from both A and B. In addition to ensuring that both ranges are strictly weakly ordered, we also need to ensure that the element types of both ranges are somehow compatible.

Thanks to the fact that it is already `concat_view` in the standard, the `concatable` concept is perfect for such a purpose. The signatures of both would be:

```
template<view V1, view V2>
    requires input_range<V1> && input_range<V2> &&
               indirect_strict_weak_order<ranges::less, iterator_t<V1>, iterator_t<V2>> &&
               concatable<V1, V2>
    class set_[union|symmetric_difference]_view;
```

Iterator design for `set_[intersection|difference]_view`

The iterators of the four views all follow a similar design. When they are constructed, we first find the next valid element through the `satisfy()` function, and then in each `operator++()`, we increment the underlying iterator and call the `satisfy()` again to find the next valid element, and so on.

Since in `satisfy()`, we also need to check whether the iterators of A or B have reached the end to determine the next valid element, we need to know the information of the sentinels of both, which means we need to store both sentinels in the iterator.

For `set_intersection_view`, its iterator signature is as follows:

```
class set_intersection_view::iterator {
    iterator_t<V1> current1_;
    sentinel_t<V1> end1_;
    iterator_t<V2> current2_;
    sentinel_t<V2> end2_;

    constexpr void
    satisfy() {
        while (current1_ != end1_ && current2_ != end2_) {
            /* Find the next valid element in the first range */
        }
    }
}
```

```

}

constexpr iterator(iterator_t<V1> current1, sentinel_t<V2> end1,
                  iterator_t<V1> current2, sentinel_t<V2> end2)
: current1_(std::move(current1)), end1_(end1),
  current2_(std::move(current2)), end2_(end2) {
    satisfy();
}

public:
constexpr decltype(auto) operator*() const { return *current1_; }

constexpr iterator&
operator++() {
    ++current1_;
    ++current2_;
    satisfy();
    return *this;
}

friend constexpr bool
operator==(const iterator& x, default_sentinel_t) {
    return x.current1_ == x.end1_ || x.current2_ == x.end2_;
}
};

```

We can slightly modify the logic of the three functions above, `satisfy()`, `operator++()`, and `operator==()`, to make a corresponding iterator for `set_difference_view`:

```

class set_difference_view::iterator {
    iterator_t<V1> current1_;
    sentinel_t<V1> end1_;
    iterator_t<V2> current2_;
    sentinel_t<V2> end2_;

    constexpr void
    satisfy() {
        while (current1_ != end1_ && current2_ != end2_) {
            /* New condition to find the next valid element in the first range */
        }
    }

    constexpr iterator(iterator_t<V1> current1, sentinel_t<V2> end1,
                      iterator_t<V1> current2, sentinel_t<V2> end2)
    : current1_(std::move(current1)), end1_(end1),
      current2_(std::move(current2)), end2_(end2) {
        satisfy();
    }

public:
constexpr decltype(auto) operator*() const { return *current1_; }

constexpr iterator&
operator++() {
    ++current1_;
++current2_;
    satisfy();
    return *this;
}

friend constexpr bool
operator==(const iterator& x, default_sentinel_t) {
    return x.current1_ == x.end1_ || x.current2_ == x.end2_;
}
};

```

Iterator design for `set_[union|symmetric_difference]_view`

For `set_union_view`'s iterator, it is necessary to know which underlying iterator is active right now, so an additional flag is need to indicate that.

In addition, since the resulting set contains elements from two different ranges, the new reference type needs to be a common reference of the two, in which case `concat-reference-t` nicely fits the purpose:

```

class set_union_view::iterator {
    iterator_t<V1> current1_;
    sentinel_t<V1> end1_;
    iterator_t<V2> current2_;
    sentinel_t<V2> end2_;
    bool use_first_;

```

```
constexpr void
satisfy() {
    /* Find the next valid element from two ranges */
}

constexpr iterator(iterator_t<V1> current1, sentinel_t<V2> end1,
                  iterator_t<V1> current2, sentinel_t<V2> end2)
    : current1_(std::move(current1)), end1_(end1),
      current2_(std::move(current2)), end2_(end2) {
    satisfy();
}

public:
constexpr concat-reference-t<V1, V2>
operator*() const {
    if (use_first_)
        return *current1_;
    return *current2_;
}

constexpr iterator&
operator++() {
    if (use_first_)
        ++current1_;
    else
        ++current2_;
    satisfy();
    return *this;
}

friend constexpr bool
operator==(const iterator& x, default_sentinel_t) {
    return x.current1_ == x.end1_ && x.current2_ == x.end2_;
}
};
```

Similarly, we can make `set_symmetric_difference_view`'s iterator by modifying `satisfy()` to skip the invalid part.

Borrowed ranges

`set_operation_view::iterator` only store iterator-sentinel pairs of two underlying views, so it is a `borrowed_range` when both models `borrowed_range`.

Non-common ranges

These view classes already have the origin sentinel stored as a member in its iterator, so their `end()` can simply return `default_sentinel`, which makes them always non-common ranges. There is no need to compromise compilation and runtime performance to just support common ranges.

Only-forward ranges

Although supporting bidirectional ranges is theoretically possible, it would introduce potential use-after-move issues due to value comparisons between the two ranges. For example, applying `views::reverse` | `views::as_rvalue` with it is an undefined behavior.

The author does not plan to support this, this is consistent with [range/v3](#).

Non const-iterable

Except for set union, `satisfy()` for other operations has the worst complexity of $O(n)$, because we need to skip the invalid white area to locate the next valid element. In order to ensure the amortized constant time complexity of `begin()` required by the `range` concept, we need to cache the iterator in the first call to `begin()`, which means that other view classes except `set_union_view` are not `const-iterable`.

However, it is worth noting that when A or B is only an input range, those views will be an input range whose `begin()` is only allowed to be called once, in which case no cache is needed, making it feasible to provide `const begin`.

Given that the current standard does not support `const-iterable` in such an aggressive way, for example `filter_view` and `drop_while_view` do not choose to do so, the author does not provide such support to be consistent.

Here is the summary table:

View	const-Iterable	Caches begin()	Complexity
<code>set_difference_view</code>	✗	✓	Amortized constant

View	const-Iterable	Caches begin()	Complexity
set_intersection_view	✗	✓	Amortized constant
set_union_view	✓	✗	Constant
set_symmetric_difference_view	✗	✓	Amortized constant

No customized comparison and projection

Although those views in range/v3 also support customized comparison and projection, i.e., `views::set_operations(rng1, rng2, pred, proj1, proj2)`, the author does not think this is a good idea. This is inconsistent with the standard design guideline.

Custom comparisons and projections should be associated with constraint algorithms rather than range adaptors. In fact, the standard never expected any range adaptor to support this, even though some of them may be doable, such as `filter_view` or `chunk_by_view`, which only support custom predicates and do not extend support for projections. As for `split_view`, the standard also explicitly specifies `ranges::equal_to` as the comparison function, leaving no room for custom specification.

Because this is an expensive tradeoff, as it requires three extra fields per iterator to store the functions. Additionally, we cannot store these functions in the view because this prevents the view from being borrowed, which is not ideal. Why bother ourselves if there are more consistent ways with `views::transform` to do the equivalent?

Also, supporting such unnecessary flexibility can bring confusion. For example, it might be unclear what `views::set_operation(rng1, x)` means, since `x` could be either a range or a predicate.

Not sure why range/v3 was designed this way, but the authors strongly prefer not to support it and intuitively chose `ranges::less` for comparing.

No reserve_hint member

While we could easily get an upper bound on the resulting view size to provide a `reserve_hint` member, the actual size value depends heavily on the sizes of A and B and the actual value of their elements, which means it could differ too much from the upper bound. In this case, the author chose not to provide a `reserve_hint` member.

No customized iter_swap specializations

It doesn't make sense to provide `iter_swap` specializations for these new iterators, since we'd break the origin order by swapping the elements which leads to undefined behavior.

Check for sorted ranges

Set operations require both ranges to be sorted, so a precondition needs to be imposed in the constructors to ensure correct semantics.

Provide base member

`set_[intersection|difference]_view` can be seen as a subset of the first range, so it makes sense to provide `base()` members for it and its iterator to access the first underlying view and iterator.

Implementation experience

The author implemented four `views::set_operations`s based on libstdc++, see [godbolt](#).

Wording

This wording is relative to the [latest working draft](#).

1. Add one new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define cpp lib_ranges_set_view 20XXXXL // freestanding, also in <ranges>
```

2. Modify 25.2 [\[ranges.syn\]](#), Header `<ranges>` synopsis, as indicated:

```
#include <compare>           // see [compare.syn]
#include <initializer_list>   // see [initializer.list.syn]
#include <iterator>           // see [iterator.synopsis]

namespace std::ranges {
    [...]
    namespace views { inline constexpr unspecified to_input = unspecified; }
```

```

// [range.set.difference], set difference view
template<view V1, view V2>
    requires see below
    class set_difference_view;

template<view V1, view V2>
    constexpr bool enable_borrowed_range<set_difference_view<V1, V2>> =
        enable_borrowed_range<V1> && enable_borrowed_range<V2>;

namespace views { inline constexpr unspecified set_difference = unspecified; }

// [range.set.intersection], set intersection view
template<view V1, view V2>
    requires see below
    class set_intersection_view;

template<view V1, view V2>
    constexpr bool enable_borrowed_range<set_intersection_view<V1, V2>> =
        enable_borrowed_range<V1> && enable_borrowed_range<V2>;

namespace views { inline constexpr unspecified set_intersection = unspecified; }

// [range.set.union], set union view
template<view V1, view V2>
    requires see below
    class set_union_view;

template<view V1, view V2>
    constexpr bool enable_borrowed_range<set_union_view<V1, V2>> =
        enable_borrowed_range<V1> && enable_borrowed_range<V2>;

namespace views { inline constexpr unspecified set_union = unspecified; }

// [range.set.symmetric.difference], set symmetric difference view
template<view V1, view V2>
    requires see below
    class set_symmetric_difference_view;

template<view V1, view V2>
    constexpr bool enable_borrowed_range<set_symmetric_difference_view<V1, V2>> =
        enable_borrowed_range<V1> && enable_borrowed_range<V2>;

namespace views { inline constexpr unspecified set_symmetric_difference = unspecified; }
}

```

3. Add **25.7.2 Set difference view** [[range.set.difference](#)] after 25.7.35 [[range.to.input](#)] as indicated:

[25.7.2.1] Overview [[range.set.difference.overview](#)]

-1- [set_difference_view](#) presents a view of set difference between two sorted ranges.

-2- The name `views::set_difference` denotes a range adaptor object (25.7.2 [[range.adaptor.object](#)]). Given subexpressions `E` and `F`, the expression `views::set_difference(E, F)` is expression-equivalent to `set_difference_view(E, F)`.

-3- [Example 1]:

```

vector v1{1, 2, 5, 5, 5, 9};
vector v2{ 2, 5, 7};
println("{} ", views::set_difference(v1, v2)); // prints [1, 5, 5, 9]

```

— end example]

[25.7.2.2] Class template [set_difference_view](#) [[range.set.difference.view](#)]

```

namespace std::ranges {
    template<class R1, class R2>
        concept set_operable = input_range<R1> && input_range<R2> && // exposition only
            indirect_strict_weak_order<ranges::less, iterator_t<R1>, iterator_t<R2>>;

    template<view V1, view V2>
        requires set_operable<V1, V2>
        class set_difference_view : public view_interface<set_difference_view<V1, V2>> {
            V1 base1_ = V1(); // exposition only
            V2 base2_ = V2(); // exposition only

            // [range.set.difference.iterator], class set_difference_view::iterator
            class iterator; // exposition only

        public:

```

```

    set_difference_view()
        requires default_initializable<V1> && default_initializable<V2> = default;

constexpr explicit set_difference_view(V1 base1, V2 base2);

constexpr V1 base() const & requires copy_constructible<V1> { return base1_; }
constexpr V1 base() && { return std::move(base1_); }

constexpr iterator begin();

constexpr default_sentinel_t end() const noexcept { return default_sentinel; }
};

template<class R1, class R2>
    set_difference_view(R1&&, R2&&)
        -> set_difference_view<views::all_t<R1>, views::all_t<R2>>;
}

constexpr explicit set_difference_view(V1 base1, V2 base2);

```

-1- *Effects*: Initializes *base1_* with `std::move(base1)` and *base2_* with `std::move(base2)`. The behavior is undefined if either `ranges::is_sorted(base1_)` or `ranges::is_sorted(base2_)` is false.

```
constexpr iterator begin();
```

-2- *Returns*: `{ranges::begin(base1_), ranges::end(base1_), ranges::begin(base2_), ranges::end(base2_)}`.

-3- *Remarks*: In order to provide the amortized constant time complexity required by the `range` concept when `set_difference_view` models `forward_range`, this function caches the result within the `set_difference_view` for use on subsequent calls.

[25.7.7.2] Class `set_difference_view::iterator` [range.set.difference.iterator]

```

namespace std::ranges {
    template<view V1, view V2>
        requires set_operable<V1, V2>
        class set_difference_view<V1, V2>::iterator {
            iterator_t<V1> current1_ = iterator_t<V1>(); // exposition only
            sentinel_t<V1> end1_ = sentinel_t<V1>(); // exposition only
            iterator_t<V2> current2_ = iterator_t<V2>(); // exposition only
            sentinel_t<V2> end2_ = sentinel_t<V2>(); // exposition only

            constexpr void satisfy(); // exposition only

            constexpr iterator(iterator_t<V1> current1, sentinel_t<V1> end1, // exposition only
                               iterator_t<V2> current2, sentinel_t<V2> end2);

        public:
            using iterator_category = see below; // not always present
            using iterator_concept =
                conditional_t<forward_range<V1> && forward_range<V2>, forward_iterator_tag, input_iterator_tag>;
            using value_type = range_value_t<V1>;
            using difference_type = range_difference_t<V1>;

            iterator()
                requires default_initializable<iterator_t<V1>> && default_initializable<iterator_t<V2>>
                = default;

            constexpr iterator_t<V1> base() && { return std::move(current1_); }
            constexpr const iterator_t<V1>& base() const & noexcept { return current1_; }

            constexpr decltype(auto) operator*() const { return *current1_; }

            constexpr iterator& operator++();
            constexpr void operator++(int);
            constexpr iterator operator++(int) requires forward_range<V1> && forward_range<V2>;

            friend constexpr bool operator==(const iterator& x, const iterator& y)
                requires equality_comparable<iterator_t<V1>>;
            friend constexpr bool operator==(const iterator& x, default_sentinel_t);

            friend constexpr decltype(auto) iter_move(const iterator& i)
                noexcept(noexcept(ranges::iter_move(i.current1_)));
        };
}

```

-1- The member *typedef-name* `iterator_category` is defined if and only if *V1* and *V2* model `forward_range`. In that case, `iterator_category` denotes `forward_iterator_tag` if *qualified-id* `iterator_traits<iterator_t<V1>>::iterator_category` and

`iterator_traits<iterator_t<V2>>::iterator_category` denote a type that models `derived_from<forward_iterator_tag>`; otherwise it denotes `input_iterator_tag`.

`constexpr void satisfy();`

-2- *Effects*: Equivalent to:

```
while (true) {
    if (current1_ == end1_)
        return;
    if (current2_ == end2_)
        return;
    if (*current1_ < *current2_)
        return;
    if (*current2_ < *current1_)
        ++current2_;
    else {
        ++current1_;
        ++current2_;
    }
}
```

[*Note 1*: `set_difference_view` iterators use the `satisfy` function to find the next valid element in the first range.—*end note*]

`constexpr iterator(iterator_t<V1> current1, sentinel_t<V1> end1,
 iterator_t<V2> current2, sentinel_t<V2> end2);`

-3- *Effects*: Initializes `current1_` with `std::move(current1)`, `end1_` with `end1`, `current2_` with `std::move(current2)`, `end2_` with `end2`; then calls `satisfy()`.

`constexpr iterator& operator++();`

-4- *Effects*: Equivalent to:

```
++current1_;
satisfy();
return *this;
```

`constexpr void operator++(int);`

-5- *Effects*: Equivalent to `++*this`.

`constexpr iterator operator++(int)` requires `forward_range<V1> && forward_range<V2>`;

-6- *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

`friend constexpr bool operator==(const iterator& x, const iterator& y)`
requires `equality_comparable<iterator_t<V1>>`;

-7- *Effects*: Equivalent to: `return x.current1_ == y.current1_;`

`friend constexpr bool operator==(const iterator& x, default_sentinel_t);`

-8- *Effects*: Equivalent to: `return x.current1_ == x.end1_;`

`friend constexpr decltype(auto) iter_move(const iterator& i)`
`noexcept(noexcept(ranges::iter_move(i.current1_)));`

-9- *Effects*: Equivalent to: `return ranges::iter_move(i.current1_);`

4. Add **25.7.? Set intersection view** [`range.set.intersection`] after [`range.set.difference`] as indicated:

[25.7.?.1] Overview [`range.set.intersection.overview`]

-1- `set_intersection_view` presents a view of set intersection between two sorted ranges.

-2- The name `views::set_intersection` denotes a range adaptor object (25.7.2 [`range.adaptor.object`]). Given subexpressions `E` and `F`, the expression `views::set_intersection(E, F)` is expression-equivalent to `set_intersection_view(E, F)`.

-3- [*Example 1*:

```
vector v1{1, 2, 2, 3, 4, 5, 6};
vector v2{ 2, 2, 3, 3, 5, 7};
```



```
println("{} ", views::set_intersection(v1, v2)); // prints [2 2 3 5]
```

— end example]

[25.7.?.2] Class template `set_intersection_view` [range.set.intersection.view]

```
namespace std::ranges {
    template<view V1, view V2>
        requires set_operable<V1, V2>
    class set_intersection_view : public view_interface<set_intersection_view<V1, V2>> {
        V1 base1_ = V1(); // exposition only
        V2 base2_ = V2(); // exposition only

        // [range.set.intersection.iterator], class set_intersection_view::iterator
        class iterator; // exposition only

    public:
        set_intersection_view()
            requires default_initializable<V1> && default_initializable<V2> = default;

        constexpr explicit set_intersection_view(V1 base1, V2 base2);

        constexpr V1 base() const & requires copy_constructible<V1> { return base1_; }
        constexpr V1 base() && { return std::move(base1_); }

        constexpr iterator begin();

        constexpr default_sentinel_t end() const noexcept { return default_sentinel; }
    };

    template<class R1, class R2>
        set_intersection_view(R1&&, R2&&)
            -> set_intersection_view<views::all_t<R1>, views::all_t<R2>>;
}
```

```
constexpr explicit set_intersection_view(V1 base1, V2 base2);
```

-1- *Effects*: Initializes `base1_` with `std::move(base1)` and `base2_` with `std::move(base2)`. The behavior is undefined if either `ranges::is_sorted(base1_)` or `ranges::is_sorted(base2_)` is false.

```
constexpr iterator begin();
```

-2- *Returns*: {`ranges::begin(base1_)`, `ranges::end(base1_)`, `ranges::begin(base2_)`, `ranges::end(base2_)`}.

-3- *Remarks*: In order to provide the amortized constant time complexity required by the `range` concept when `set_intersection_view` models `forward_range`, this function caches the result within the `set_intersection_view` for use on subsequent calls.

[25.7.?.2] Class `set_intersection_view::iterator` [range.set.intersection.iterator]

```
namespace std::ranges {
    template<view V1, view V2>
        requires set_operable<V1, V2>
    class set_intersection_view<V1, V2>::iterator {
        iterator_t<V1> current1_ = iterator_t<V1>(); // exposition only
        sentinel_t<V1> end1_ = sentinel_t<V1>(); // exposition only
        iterator_t<V2> current2_ = iterator_t<V2>(); // exposition only
        sentinel_t<V2> end2_ = sentinel_t<V2>(); // exposition only

        constexpr void satisfy(); // exposition only

        constexpr iterator(iterator_t<V1> current1, sentinel_t<V1> end1, // exposition only
                           iterator_t<V2> current2, sentinel_t<V2> end2);

    public:
        using iterator_category = see below; // not always present
        using iterator_concept =
            conditional_t<forward_range<V1> && forward_range<V2>, forward_iterator_tag, input_iterator_tag>;
        using value_type = range_value_t<V1>;
        using difference_type = range_difference_t<V1>;

        iterator()
            requires default_initializable<iterator_t<V1>> && default_initializable<iterator_t<V2>>
            = default;

        constexpr iterator_t<V1> base() && { return std::move(current1_); }
        constexpr const iterator_t<V1>& base() const & noexcept { return current1_; }

        constexpr decltype(auto) operator*() const { return *current1_; }
    };
}
```

```
constexpr iterator& operator++();
constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<V1> && forward_range<V2>;

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<V1>>;
friend constexpr bool operator==(const iterator& x, default_sentinel_t);

friend constexpr decltype(auto) iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current1_)));
};
}
```

-1- The member *typedef-name* `iterator_category` is defined if and only if `V1` and `V2` model `forward_range`. In that case, `iterator_category` denotes `forward_iterator_tag` if *qualified-id* `iterator_traits<iterator_t<V1>>::iterator_category` and `iterator_traits<iterator_t<V2>>::iterator_category` denote a type that models `derived_from<forward_iterator_tag>`; otherwise it denotes `input_iterator_tag`.

```
constexpr void satisfy();
```

-2- *Effects*: Equivalent to:

```
while (true) {
    if (current1_ == end1_)
        return;
    if (current2_ == end2_)
        return;
    if (*current1_ < *current2_)
        ++current1_;
    else if (*current2_ < *current1_)
        ++current2_;
    else
        return;
}
```

[*Note 1*: `set_intersection_view` iterators use the `satisfy` function to find the next valid element in the first range.—*end note*]

```
constexpr iterator(iterator_t<V1> current1, sentinel_t<V1> end1,
                  iterator_t<V2> current2, sentinel_t<V2> end2);
```

-3- *Effects*: Initializes `current1_` with `std::move(current1)`, `end1_` with `end1`, `current2_` with `std::move(current2)`, `end2_` with `end2`; then calls `satisfy()`.

```
constexpr iterator& operator++();
```

-4- *Effects*: Equivalent to:

```
++current1_;
++current2_;
satisfy();
return *this;
```

```
constexpr void operator++(int);
```

-5- *Effects*: Equivalent to `++*this`.

```
constexpr iterator operator++(int) requires forward_range<V1> && forward_range<V2>;
```

-6- *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<V1>>;
```

-7- *Effects*: Equivalent to: `return x.current1_ == y.current1_;`

```
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
```

-8- *Effects*: Equivalent to: `return x.current1_ == x.end1_ || x.current2_ == x.end2_;`

```
friend constexpr decltype(auto) iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current1_)));
```

-9- *Effects*: Equivalent to: `return ranges::iter_move(i.current1_);`

5. Add **25.7.2 Set union view** [[range.set.union](#)] after [[range.set.intersection](#)] as indicated:

[25.7.2.1] Overview [[range.set.union.overview](#)]

-1- `set_union_view` presents a view of set union between two sorted ranges.

-2- The name `views::set_union` denotes a range adaptor object (25.7.2 [[range.adaptor.object](#)]). Given subexpressions `E` and `F`, the expression `views::set_union(E, F)` is expression-equivalent to `set_union_view(E, F)`.

-3- [Example 1:

```
vector v1{1, 2, 3, 4, 5};
vector v2{    3, 4, 5, 6, 7};
println("{} ", views::set_union(v1, v2)); // prints [1 2 3 4 5 6 7]
```

— end example]

[25.7.2.2] Class template `set_union_view` [[range.set.union.view](#)]

```
namespace std::ranges {
    template<class R1, class R2>
        concept set_operable_concatable =          // exposition only
            set_operable<R1, R2> && concatable<R1, R2>;

    template<view V1, view V2>
        requires set_operable_concatable<V1, V2>
    class set_union_view : public view_interface<set_union_view<V1, V2>> {
        V1 base1_ = V1();           // exposition only
        V2 base2_ = V2();           // exposition only

        // [range.set.union.iterator], class template set_union_view::iterator
        template<bool Const>
        class iterator;              // exposition only

    public:
        set_union_view()
            requires default_initializable<V1> && default_initializable<V2> = default;

        constexpr explicit set_union_view(V1 base1, V2 base2);

        constexpr auto begin() requires (!simple_view<V1> || !simple_view<V2>) {
            return iterator<false>(ranges::begin(base1_), ranges::end(base1_),
                                   ranges::begin(base2_), ranges::end(base2_));
        }

        constexpr auto begin() const requires set_operable_concatable<V1, V2> {
            return iterator<true>(ranges::begin(base1_), ranges::end(base1_),
                                  ranges::begin(base2_), ranges::end(base2_));
        }

        constexpr default_sentinel_t end() const noexcept { return default_sentinel; }
    };

    template<class R1, class R2>
        set_union_view(R1&&, R2&&)
            -> set_union_view<views::all_t<R1>, views::all_t<R2>>;
}
```

```
constexpr explicit set_union_view(V1 base1, V2 base2);
```

-1- *Effects*: Initializes `base1_` with `std::move(base1)` and `base2_` with `std::move(base2)`. The behavior is undefined if either `ranges::is_sorted(base1_)` or `ranges::is_sorted(base2_)` is `false`.

[25.7.2.2] Class template `set_union_view::iterator` [[range.set.union.iterator](#)]

```
namespace std::ranges {
    template<view V1, view V2>
        requires set_operable_concatable<V1, V2>
    template<bool Const>
    class set_union_view<V1, V2>::iterator {
        using Base1 = maybe_const<Const, V1>;           // exposition only
        using Base2 = maybe_const<Const, V2>;           // exposition only
        iterator_t<Base1> current1_ = iterator_t<Base1>(); // exposition only
        sentinel_t<Base1> end1_ = sentinel_t<Base1>();   // exposition only
        iterator_t<Base2> current2_ = iterator_t<Base2>(); // exposition only
        sentinel_t<Base2> end2_ = sentinel_t<Base2>();   // exposition only
        bool use_first_ = false;                         // exposition only

        constexpr void satisfy();                        // exposition only

        constexpr iterator(iterator_t<Base1> current1, sentinel_t<Base1> end1, // exposition only
```

```

        iterator_t<Base2> current2, sentinel_t<Base2> end2);

public:
    using iterator_category = see below;                // not always present
    using iterator_concept =
        conditional_t<forward_range<Base1> && forward_range<Base2>, forward_iterator_tag, input_iterator_tag>;
    using value_type = concat_value_t<V1, V2>;
    using difference_type = common_type_t<range_difference_t<Base1>, range_difference_t<Base2>>;

    iterator()
        requires default_initializable<iterator_t<Base1>> && default_initializable<iterator_t<Base2>>
        = default;

    constexpr iterator(iterator<!Const> i)
        requires Const && convertible_to<iterator_t<V1>, iterator_t<Base1>> &&
            convertible_to<sentinel_t<V1>, sentinel_t<Base1>> &&
            convertible_to<iterator_t<V2>, iterator_t<Base2>> &&
            convertible_to<sentinel_t<V2>, sentinel_t<Base2>>;

    constexpr concat_reference_t<Base1, Base2> operator*() const;

    constexpr iterator& operator++();
    constexpr void operator++(int);
    constexpr iterator operator++(int) requires forward_range<Base1> && forward_range<Base2>;

    friend constexpr bool operator==(const iterator& x, const iterator& y)
        requires equality_comparable<iterator_t<Base1>> && equality_comparable<iterator_t<Base2>>;
    friend constexpr bool operator==(const iterator& x, default_sentinel_t);

    friend constexpr concat_rvalue_reference_t<Base1, Base2> iter_move(const iterator& i) noexcept(see below);
};

```

-1- The member *typedef-name* `iterator_category` is defined if and only if `Base1` and `Base2` model `forward_range`. In that case, `iterator_category` denotes `forward_iterator_tag` if *qualified-id* `iterator_traits<iterator_t<Base1>>::iterator_category` and `iterator_traits<iterator_t<Base2>>::iterator_category` denote a type that models `derived_from<forward_iterator_tag>`; otherwise it denotes `input_iterator_tag`.

```
constexpr void satisfy();
```

-2- *Effects*: Equivalent to:

```

if (current1_ == end1_) {
    use_first_ = false;
    return;
}
if (current2_ == end2_) {
    use_first_ = true;
    return;
}
if (*current1_ < *current2_) {
    use_first_ = true;
    return;
}
if (*current2_ < *current1_) {
    use_first_ = false;
    return;
}
use_first_ = true;
++current2_;

```

[Note 1: `set_union_view` iterators use the `satisfy` function to find the next valid element in two ranges.— *end note*]

```
constexpr iterator(iterator_t<Base1> current1, sentinel_t<Base1> end1,
    iterator_t<Base2> current2, sentinel_t<Base2> end2);
```

-3- *Effects*: Initializes `current1_` with `std::move(current1)`, `end1_` with `end1`, `current2_` with `std::move(current2)`, `end2_` with `end2`; then calls `satisfy()`.

```

constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V1>, iterator_t<Base1>> &&
        convertible_to<sentinel_t<V1>, sentinel_t<Base1>> &&
        convertible_to<iterator_t<V2>, iterator_t<Base2>> &&
        convertible_to<sentinel_t<V2>, sentinel_t<Base2>>;

```

-4- *Effects*: Initializes `current1_` with `std::move(i.current1_)`, `end1_` with `i.end1_`, `current2_` with `std::move(i.current2_)`, `end2_` with `i.end2_`, and `use_first_` with `i.use_first_`.

```
constexpr concat_reference_t<Base1, Base2> operator*() const;
```

-5- *Effects*: Equivalent to:

```
if (use_first_)
    return *current1_;
return *current2_;
```

```
constexpr iterator& operator++();
```

-6- *Effects*: Equivalent to:

```
if (use_first_)
    ++current1_;
else
    ++current2_;
satisfy();
return *this;
```

```
constexpr void operator++(int);
```

-7- *Effects*: Equivalent to `++*this`.

```
constexpr iterator operator++(int) requires forward_range<Base1> && forward_range<Base2>;
```

-8- *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<Base1>> && equality_comparable<iterator_t<Base2>>;
```

-9- *Effects*: Equivalent to: `return x.current1_ == y.current1_ && x.current2_ == y.current2_;`

```
friend constexpr bool operator==(const iterator& x, default_sentinel_t);
```

-10- *Effects*: Equivalent to: `return x.current1_ == x.end1_ && x.current2_ == x.end2_;`

```
friend constexpr concat_rvalue_reference_t<Base1, Base2> iter_move(const iterator& i) noexcept(see below);
```

-11- *Effects*: Equivalent to:

```
if (i.use_first_)
    return ranges::iter_move(i.current1_);
return ranges::iter_move(i.current2_);
```

-12- *Remarks*: The exception specification is equivalent to:

```
noexcept(ranges::iter_move(i.current1_)) &&
noexcept(ranges::iter_move(i.current2_)) &&
is_nothrow_convertible_v<range_rvalue_reference_t<Base1>, concat_rvalue_reference_t<Base1, Base2>> &&
is_nothrow_convertible_v<range_rvalue_reference_t<Base2>, concat_rvalue_reference_t<Base1, Base2>>
```

Add **25.7.2 Set symmetric difference view** [`range.set.symmetric.difference`] after [`range.set.union`] as indicated:

[25.7.2.1] Overview [`range.set.symmetric.difference.overview`]

-1- `set_symmetric_difference_view` presents a view of set symmetric difference between two sorted ranges.

-2- The name `views::set_symmetric_difference` denotes a range adaptor object (25.7.2 [[range.adaptor.object](#)]). Given subexpressions `E` and `F`, the expression `views::set_symmetric_difference(E, F)` is expression-equivalent to `set_symmetric_difference_view(E, F)`.

-3- [Example 1:

```
vector v1{1, 3, 4, 6, 7, 9};
vector v2{1, 4, 5, 6, 9};
println("{} ", views::set_symmetric_difference(v1, v2)); // prints [3 5 7]
```

— end example]

[25.7.2.2] Class template `set_symmetric_difference_view` [`range.set.symmetric.difference.view`]

```
namespace std::ranges {
    template<view V1, view V2>
        requires set_operable_concatable<V1, V2>
        class set_symmetric_difference_view : public view_interface<set_symmetric_difference_view<V1, V2>> {
```

```

V1 base1_ = V1();           // exposition only
V2 base2_ = V2();           // exposition only

// [range.set.symmetric.difference.iterator], class set_symmetric_difference_view::iterator
class iterator;             // exposition only

public:
    set_symmetric_difference_view()
        requires default_initializable<V1> && default_initializable<V2> = default;

    constexpr explicit set_symmetric_difference_view(V1 base1, V2 base2);

    constexpr iterator begin();

    constexpr default_sentinel_t end() const noexcept { return default_sentinel; }
};

template<class R1, class R2>
    set_symmetric_difference_view(R1&&, R2&&)
        -> set_symmetric_difference_view<views::all_t<R1>, views::all_t<R2>>;
}

constexpr explicit set_symmetric_difference_view(V1 base1, V2 base2);

-1- Effects: Initializes base1_ with std::move(base1) and base2_ with std::move(base2). The behavior is undefined
if either ranges::is_sorted(base1_) or ranges::is_sorted(base2_) is false.

constexpr iterator begin();

-2- Returns: {ranges::begin(base1_), ranges::end(base1_), ranges::begin(base2_),
ranges::end(base2_)}.

-3- Remarks: In order to provide the amortized constant time complexity required by the range concept when
set_symmetric_difference_view models forward_range, this function caches the result within the
set_symmetric_difference_view for use on subsequent calls.

```

[25.7.?.2] Class `set_symmetric_difference_view::iterator` [range.set.symmetric.difference.iterator]

```

namespace std::ranges {
    template<view V1, view V2>
        requires set_operable_concatable<V1, V2>
        class set_symmetric_difference_view<V1, V2>::iterator {
            iterator_t<V1> current1_ = iterator_t<V1>();           // exposition only
            sentinel_t<V1> end1_     = sentinel_t<V1>();           // exposition only
            iterator_t<V2> current2_ = iterator_t<V2>();           // exposition only
            sentinel_t<V2> end2_     = sentinel_t<V2>();           // exposition only
            bool use_first_ = false;                                // exposition only

            constexpr void satisfy();                                // exposition only

            constexpr iterator(iterator_t<V1> current1, sentinel_t<V1> end1,           // exposition only
                               iterator_t<V2> current2, sentinel_t<V2> end2);

        public:
            using iterator_category = see below;                    // not always present
            using iterator_concept =
                conditional_t<forward_range<V1> && forward_range<V2>, forward_iterator_tag, input_iterator_tag>;
            using value_type       = concat_value_t<V1, V2>;
            using difference_type   = common_type_t<range_difference_t<V1>, range_difference_t<V2>>;

            iterator()
                requires default_initializable<iterator_t<V1>> && default_initializable<iterator_t<V2>>
                = default;

            constexpr concat_reference_t<V1, V2> operator*() const;

            constexpr iterator& operator++();
            constexpr void operator++(int);
            constexpr iterator operator++(int) requires forward_range<V1> && forward_range<V2>;

            friend constexpr bool operator==(const iterator& x, const iterator& y)
                requires equality_comparable<iterator_t<V1>> && equality_comparable<iterator_t<V2>>;
            friend constexpr bool operator==(const iterator& x, default_sentinel_t);

            friend constexpr concat_rvalue_reference_t<V1, V2> iter_move(const iterator& i) noexcept(see below);
        };
}

```

-1- The member *typedef-name* `iterator_category` is defined if and only if *V1* and *V2* model `forward_range`. In that case, `iterator_category` denotes `forward_iterator_tag` if *qualified-id* `iterator_traits<iterator_t<V1>>::iterator_category` and

`iterator_traits<iterator_t<V2>>::iterator_category` denote a type that models `derived_from<forward_iterator_tag>`; otherwise it denotes `input_iterator_tag`.

`constexpr void satisfy();`

-2- *Effects*: Equivalent to:

```
while (true) {
    if (current1_ == end1_) {
        use_first_ = false;
        return;
    }
    if (current2_ == end2_) {
        use_first_ = true;
        return;
    }
    if (*current1_ < *current2_) {
        use_first_ = true;
        return;
    }
    if (*current2_ < *current1_) {
        use_first_ = false;
        return;
    }
    ++current1_;
    ++current2_;
}
```

[Note 1: `set_symmetric_difference_view` iterators use the *satisfy* function to find the next valid element in two ranges.— end note]

`constexpr iterator(iterator_t<V1> current1, sentinel_t<V1> end1,
 iterator_t<V2> current2, sentinel_t<V2> end2);`

-3- *Effects*: Initializes `current1_` with `std::move(current1)`, `end1_` with `end1`, `current2_` with `std::move(current2)`, `end2_` with `end2`; then calls *satisfy*().

`constexpr concat-reference-t<V1, V2> operator*() const;`

-4- *Effects*: Equivalent to:

```
if (use_first_)
    return *current1_;
return *current2_;
```

`constexpr iterator& operator++();`

-5- *Effects*: Equivalent to:

```
if (use_first_)
    ++current1_;
else
    ++current2_;
satisfy();
return *this;
```

`constexpr void operator++(int);`

-6- *Effects*: Equivalent to `++*this`.

`constexpr iterator operator++(int)` requires `forward_range<V1>` && `forward_range<V2>`;

-7- *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

`friend constexpr bool operator==(const iterator& x, const iterator& y)`
requires `equality_comparable<iterator_t<V1>>` && `equality_comparable<iterator_t<V2>>`;

-8- *Effects*: Equivalent to: `return x.current1_ == y.current1_ && x.current2_ == y.current2_;`

`friend constexpr bool operator==(const iterator& x, default_sentinel_t);`

-9- *Effects*: Equivalent to: `return x.current1_ == x.end1_ && x.current2_ == x.end2_;`

`friend constexpr concat-rvalue-reference-t<V1, V2> iter_move(const iterator& i) noexcept(see below);`

-10- *Effects*: Equivalent to:

```
if (i.use_first_)  
    return ranges::iter_move(i.current1_);  
return ranges::iter_move(i.current2_);
```

-11- *Remarks*: The exception specification is equivalent to:

```
noexcept(ranges::iter_move(i.current1_)) &&  
noexcept(ranges::iter_move(i.current2_)) &&  
is_nothrow_convertible_v<range_rvalue_reference_t<V1>, concat-rvalue-reference-t<V1, V2>> &&  
is_nothrow_convertible_v<range_rvalue_reference_t<V2>, concat-rvalue-reference-t<V1, V2>>
```

References

[P2760R1]

Barry Revzin. A Plan for C++26 Ranges. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r1.html>

[range/v3]

Eric Niebler. `views::set_operations` implementation. URL: https://github.com/ericniebler/range-v3/blob/master/include/range/v3/view/set_algorithm.hpp